2026-8-21 Linux线程--POSIX信号量

- 1. 信号量是什么
  - 1.1 从影厅座位理解信号量
  - 1.2 P 操作与 V 操作
    - 1.2.1 P 操作：申请资源
    - 1.2.2 V 操作：归还资源
  - 1.3 信号量本身也是共享资源
- 2. 二元信号量与计数信号量
  - 2.1 二元信号量
  - 2.2 计数信号量
- 3. POSIX 信号量接口
  - 3.1 初始化信号量
  - 3.2 申请、释放与销毁
    - 3.2.1 sem_wait 与 P 操作
    - 3.2.2 sem_post 与 V 操作
    - 3.2.3 销毁信号量
- 4. 封装 Sem 类
  - 4.1 用 P/V 接口隐藏 sem_t
  - 4.2 封装时需要注意的规则
- 5. 用信号量实现环形队列
  - 5.1 环形队列的队头与队尾
  - 5.2 单生产者、单消费者环形队列
  - 5.3 生产消费流程
  - 5.4 扩展到多生产者、多消费者
  - 5.5 单生产单消费测试
- 6. POSIX 信号量的核心结论
- 7. 原始附图保留

1. 信号量是什么

信号量（semaphore）本质上是一个描述临界资源数量的计数器：

```
信号量值 > 0：表示还有若干份资源可申请
信号量值 = 0：表示资源暂时用完，申请者需要等待
归还资源后：计数器增加，可能唤醒等待者
```

这里的“资源”可以是一个空位、一份数据、一个座位或一次访问资格。信号量并不描述资源的具体内容，它只负责记录数量并协调申请与归还。

1.1 从影厅座位理解信号量

假设影厅有 100 个座位，可以把“剩余座位数”看作初始值为 100 的信号量：

- 一位观众买票，相当于申请一份座位资源，信号量减一。
- 买票成功后，这个座位就被预订。
- 如果没有余票，观众只能等待。
- 退回一张票，相当于归还一份资源，信号量加一，并可能唤醒正在等待的人。

购票的本质是对资源的提前预订。资源成功申请后，所有者就获得了使用资格；资源尚未归还时，其他申请者必须按照信号量规则等待。

1.2 P 操作与 V 操作

信号量通常通过两个原子操作使用：

1.2.1 P 操作：申请资源

P 操作可以理解为：

```
if (sem > 0) {
    sem--;
    获得资源，继续执行;
} else {
    阻塞等待;
}
```

计数大于零时，P 操作原子地减一并成功返回；计数为零时，当前线程进入等待队列，直到其他线程归还资源。

1.2.2 V 操作：归还资源

V 操作可以理解为：

```
sem++;
唤醒可能的等待者;
```

V 操作增加资源计数，并通知等待者重新参与资源竞争。P 和 V 都不会在中途被其他 P/V 操作打断，否则计数器本身就会产生数据竞争。

1.3 信号量本身也是共享资源

所有需要协作的线程必须看到同一个信号量。一个线程通过 P 操作减少了资源数量，其他线程也应该在同一个计数上观察和申请。

P 操作和 V 操作由信号量实现保证原子性，因此调用者不需要再给计数器的加减过程额外加锁。但信号量保护的业务数据是否已经安全修改，仍取决于使用顺序和临界区设计。

2. 二元信号量与计数信号量

2.1 二元信号量

信号量初始值为 `1` 时，称为二元信号量：

```
sem = 1：资源空闲
sem = 0：资源已被占用
```

第一个线程执行 P 操作后使计数变成 `0`，成功进入临界区；其他线程再执行 P 操作就会阻塞。持有者执行 V 操作使计数恢复为 `1` 后，等待者才能继续竞争。

从“同一时刻只允许一个执行流进入”的角度看，二元信号量与互斥量非常相似，也可以表示为“只有一份访问资格”。

不过 POSIX 互斥量通常要求由加锁的线程负责解锁，而普通信号量没有这种强制所有权约束：一个线程可以执行 P，另一个线程可以执行 V。因此二元信号量可以承担互斥功能，但不能不加区分地当作 `pthread_mutex_t` 的替代。

2.2 计数信号量

初始值大于 `1` 的信号量表示有多份同类资源。例如容量为 `N` 的环形队列可以设置：

```
空白位置资源 = N
已有数据资源 = 0
```

- 生产者申请一个空位：`blank_sem.P()`。
- 消费者申请一份数据：`data_sem.P()`。
- 生产者写入后归还一份数据：`data_sem.V()`。
- 消费者取走后归还一个空位：`blank_sem.V()`。

这样，空位数量和数据数量都由信号量自动维护。队列满时空位计数为零，生产者会等待；队列空时数据计数为零，消费者会等待。

3. POSIX 信号量接口

3.1 初始化信号量

POSIX 无名信号量的类型是 `sem_t`，主要接口如下：

```
#include <semaphore.h>

int sem_init(sem_t* sem, int pshared, unsigned int value);
int sem_destroy(sem_t* sem);

int sem_wait(sem_t* sem);   // P 操作
int sem_post(sem_t* sem);   // V 操作
```

`sem_init` 的三个参数：

- `sem`：要初始化的信号量对象。
- `pshared`：线程共享方式。
- `value`：信号量初始值。

通常在线程之间使用时传入 `pshared = 0`。如果需要在多个进程之间共享无名信号量，则要使用非零值，并把信号量放在进程间共享内存中。

源笔记使用的初始化方式为：

```
sem_t sem;
```

```
sem_init(&sem, 0, initial_value);
```

3.2 申请、释放与销毁

3.2.1 `sem_wait` 与 P 操作

```
int sem_wait(sem_t* sem);
```

- 信号量值大于 `0` 时，原子地减一并立即返回。
- 信号量值为 `0` 时，当前线程阻塞。
- 被唤醒后，线程重新竞争资源。

`sem_wait` 成功返回 `0`，失败返回 `-1` 并设置 `errno`。如果调用被信号中断，它可能返回失败并令 `errno == EINTR`，需要根据程序需要决定是否重试。

3.2.2 `sem_post` 与 V 操作

```
int sem_post(sem_t* sem);
```

`sem_post` 原子地增加信号量计数，并唤醒可能正在等待的线程。成功返回 `0`，失败返回 `-1` 并设置 `errno`。

3.2.3 销毁信号量

```
int sem_destroy(sem_t* sem);
```

销毁前必须确保没有线程正在等待或继续使用该信号量。对于单生产单消费的短时示例可以依赖进程退出，但正规的封装应在所有工作线程结束后再销毁。

4. 封装 `Sem` 类

4.1 用 P/V 接口隐藏 `sem_t`

源笔记用一个轻量类包装 `sem_t`，对外只暴露 `P` 和 `V`：

```cpp
#pragma once

#include <cerrno>
#include <stdexcept>
#include <semaphore.h>
#include <system_error>

class Sem {
public:
    explicit Sem(unsigned int initial_value)
    {
        if (sem_init(&sem_, 0, initial_value) == -1) {
            throw std::system_error(
                errno, std::generic_category(), "sem_init");
        }
    }

    Sem(const Sem&) = delete;
    Sem& operator=(const Sem&) = delete;

    void P()
    {
        while (sem_wait(&sem_) == -1) {
            if (errno != EINTR) {
                throw std::system_error(
                    errno, std::generic_category(), "sem_wait");
            }
        }
    }

    void V()
    {
        if (sem_post(&sem_) == -1) {
            throw std::system_error(
                errno, std::generic_category(), "sem_post");
        }
    }

    ~Sem()
    {
        sem_destroy(&sem_);
    }

private:
```

```
    sem_t sem_;
};
```

封装后的调用语义更清晰：

```
sem.P(); // 申请一个资源
sem.V(); // 归还一个资源
```

4.2 封装时需要注意的规则

- `sem_t` 不能被随意复制，否则两个对象可能错误地管理同一份内核状态。
- 初始化失败后不能继续使用该信号量。
- `sem_wait` 被信号中断时应重新尝试，而不是把一次中断误当成普通失败。
- 析构前必须保证没有线程仍在使用或等待这个信号量。
- 封装只统一接口，不会自动建立“申请哪个资源、归还哪个资源”的业务关系。

5. 用信号量实现环形队列

5.1 环形队列的队头与队尾

环形队列使用固定容量的数组或 `std::vector`：

```
head：消费者从这里取数据
tail：生产者从这里写数据
```

每次访问后循环前进：

```
head = (head + 1) % capacity;
tail = (tail + 1) % capacity;
```

环形队列必须满足：

1. 队列为空时，消费者不能在 `head` 位置取数据。
2. 队列为满时，生产者不能在 `tail` 位置写数据。
3. 生产者不能绕一圈后追上消费者。
4. 消费者不能越过生产者读取尚未写入的数据。

`head == tail` 既可能表示空，也可能表示满，因此只靠两个下标无法区分状态。信号量通过“空位数量”和“数据数量”提供了额外信息。

5.2 单生产者、单消费者环形队列

源笔记的 `RingQueue` 使用两个信号量：

- `_blank_sem`：初始值为容量，表示还有多少个空位。
- `_data_sem`：初始值为 `0`，表示已经有多少份数据。

```
#pragma once

#include <cstddef>
#include <vector>
#include "Sem.hpp"

template <typename T>
class RingQueue {
public:
    explicit RingQueue(std::size_t capacity = 5)
        : capacity_(capacity),
          ring_(capacity),
          consumer_step_(0),
          producer_step_(0),
          data_sem_(0),
          blank_sem_(capacity)
    {
    }

    RingQueue(const RingQueue&) = delete;
    RingQueue& operator=(const RingQueue&) = delete;

    void Enqueue(const T& value)
    {
        // 1. 预订一个空位
        blank_sem_.P();

        // 2. 在队尾写入，并让队尾循环前进
        ring_[producer_step_] = value;
        producer_step_ = (producer_step_ + 1) % capacity_;

        // 3. 新增一份可消费的数据
        data_sem_.V();
```

```
    }

    void Pop(T* out)
    {
        // 1. 预订一份数据
        data_sem_.P();

        // 2. 从队头读取，并让队头循环前进
        *out = ring_[consumer_step_];
        consumer_step_ = (consumer_step_ + 1) % capacity_;

        // 3. 归还一个空位
        blank_sem_.V();
    }

private:
    std::size_t capacity_;
    std::vector<T> ring_;
    std::size_t consumer_step_;
    std::size_t producer_step_;

    Sem data_sem_;  // 消费者关心
    Sem blank_sem_; // 生产者关心
};
```

成员初始化顺序与声明顺序一致，因此代码中先声明 `data_sem_`，再声明 `blank_sem_`。生产者和消费者分别修改自己的下标，在本节“单生产者、单消费者”的前提下，不需要额外的队列互斥锁。

5.3 生产消费流程

生产者先申请空位，再写数据：

```
P(blank_sem)
    ↓
ring[tail] = data
    ↓
tail = (tail + 1) % N
    ↓
V(data_sem)
```

消费者先申请数据，再取数据：

```
P(data_sem)
    ↓
out = ring[head]
    ↓
head = (head + 1) % N
    ↓
V(blank_sem)
```

P 操作保证资源确实存在，V 操作负责把新产生的资源告知对端。两个信号量的初始值分别对应“队列初始全空”和“队列初始无数据”。

5.4 扩展到多生产者、多消费者

上面的版本只适合单生产者和单消费者。存在多个生产者时，它们会同时修改 `producer_step_`；存在多个消费者时，它们会同时修改 `consumer_step_`。信号量只能协调资源数量，不能自动保护两个下标变量。

因此多线程版本还要在类中增加一把互斥锁：

```
private:
    pthread_mutex_t mutex_;
```

它应与两个信号量一起初始化，并在环形队列析构时销毁。生产者和消费者的操作改为：

```
void Enqueue(const T& value)
{
    blank_sem_.P();

    pthread_mutex_lock(&mutex_);
    ring_[producer_step_] = value;
    producer_step_ = (producer_step_ + 1) % capacity_;
    pthread_mutex_unlock(&mutex_);

    data_sem_.V();
}

void Pop(T* out)
{
    data_sem_.P();

    pthread_mutex_lock(&mutex_);
```

```
    *out = ring_[consumer_step_];
    consumer_step_ = (consumer_step_ + 1) % capacity_;
    pthread_mutex_unlock(&mutex_);

    blank_sem_.V();
}
```

互斥锁只保护环形队列的元数据和槽位读写。等待空位、等待数据时不要占着互斥锁，否则会降低并发度，严重时还可能形成循环等待。

当队列既不为空也不为满时，生产者访问 `tail` 指向的空位，消费者访问 `head` 指向的数据。两者使用不同位置，但队头、队尾的修改仍需互斥。

5.5 单生产单消费测试

源笔记的测试程序让生产者持续写入整数，消费者每隔一秒取出一个：

```
#include <iostream>
#include <unistd.h>
#include <pthread.h>
#include "RingQueue.hpp"

void* ProducerRoutine(void* args)
{
    auto* queue = static_cast<RingQueue<int>*>(args);
    int data = 1;

    while (true) {
        queue->Enqueue(data);
        std::cout << "生产:  " << data++ << std::endl;
    }
    return nullptr;
}

void* ConsumerRoutine(void* args)
{
    auto* queue = static_cast<RingQueue<int>*>(args);
    int data = 0;

    while (true) {
        sleep(1);
        queue->Pop(&data);
        std::cout << "消费:  " << data << std::endl;
```

```
    }
    return nullptr;
}

int main()
{
    RingQueue<int> queue(5);

    pthread_t consumer;
    pthread_t producer;
    pthread_create(&consumer, nullptr, ConsumerRoutine, &queue);
    pthread_create(&producer, nullptr, ProducerRoutine, &queue);

    pthread_join(consumer, nullptr);
    pthread_join(producer, nullptr);
    return 0;
}
```

生产者快速填充环形队列，直到空位信号量的值变为 `0`；消费者每取走一个数据就归还一个空位，生产者才能继续写入。

6. POSIX 信号量的核心结论

- 信号量本质上是临界资源数量的计数器。
- P 操作申请资源，计数大于零时减一，否则阻塞；V 操作归还资源、增加计数并唤醒等待者。
- P 和 V 是原子操作，但多个线程必须使用同一个信号量。
- 初始值为 `1` 的二元信号量可以表示一份访问资格，初始值大于 `1` 的信号量可以表示多份同类资源。
- POSIX 无名信号量通过 `sem_init`、`sem_wait`、`sem_post` 和 `sem_destroy` 完成生命周期及 P/V 操作。
- 环形队列用“空位信号量 + 数据信号量”协调生产与消费，`head` 和 `tail` 控制环形位置。
- 多生产者多消费者除了信号量，还需要互斥量保护队头和队尾，避免下标同时被修改。
- 信号量负责资源数量，互斥量负责共享结构安全，两者解决的问题不同，不能互相完全替代。

7. 原始附图保留

> 以下截图来自原手记，内容为时间函数文档，与本节 POSIX 信号量正文无关，仅为完整保留原笔记资料。

```
tm(3type)

NAME
       tm - broken-down time

LIBRARY
       Standard C library (libc)

SYNOPSIS
       #include <time.h>

       struct tm {
           int         tm_sec;    /* Seconds          [0, 60] */
           int         tm_min;    /* Minutes          [0, 59] */
           int         tm_hour;   /* Hour             [0, 23] */
           int         tm_mday;   /* Day of the month [1, 31] */
           int         tm_mon;    /* Month            [0, 11]  (January = 0) */
           int         tm_year;   /* Year minus 1900 */
           int         tm_wday;   /* Day of the week  [0, 6]   (Sunday = 0) */
           int         tm_yday;   /* Day of the year  [0, 365] (Jan/01 = 0) */
           int         tm_isdst;  /* Daylight savings flag */

           long        tm_gmtoff; /* Seconds East of UTC */
           const char *tm_zone;   /* Timezone abbreviation */
       };

   Feature Test Macro Requirements for glibc (see feature_test_macros(7)):

       tm_gmtoff, tm_zone:
           Since glibc 2.20:
               _DEFAULT_SOURCE
           glibc 2.20 and earlier:
               _BSD_SOURCE
```

```
ctime(3)                                                    Library Functions Manual

NAME
       asctime, ctime, gmtime, localtime, mktime, asctime_r, ctime_r, gmtime_r, localtime_r - transform date and time to broken-down time or ASCII

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <time.h>

       char *asctime(const struct tm *tm);
       char *asctime_r(const struct tm *restrict tm,
                       char buf[restrict 26]);

       char *ctime(const time_t *timep);
       char *ctime_r(const time_t *restrict timep,
                       char buf[restrict 26]);

       struct tm *gmtime(const time_t *timep);
       struct tm *gmtime_r(const time_t *restrict timep,
                       struct tm *restrict result);

       struct tm *localtime(const time_t *timep);
       struct tm *localtime_r(const time_t *restrict timep,
                       struct tm *restrict result);

       time_t mktime(struct tm *tm);
```